

BlogApp - Complete Project Documentation

Table of Contents

1. [Project Overview](#)
 2. [Project Structure](#)
 3. [Application Flow](#)
 4. [File-by-File Detailed Explanation](#)
 5. [Complete Request Flow Examples](#)
 6. [Key Concepts Explained](#)
-

Project Overview

BlogApp is a full-stack MERN (MongoDB, Express, React, Node.js) backend application that provides:

- User authentication (Register, Login, Logout)
- Blog CRUD operations (Create, Read, Update, Delete)
- JWT-based authentication
- Password hashing with bcrypt
- Error handling middleware
- CORS configuration for frontend-backend communication

Tech Stack:

- **Backend:** Node.js + Express.js
 - **Database:** MongoDB + Mongoose
 - **Authentication:** JWT (JSON Web Tokens)
 - **Security:** bcryptjs (password hashing), cookie-parser
 - **Environment Variables:** dotenv
-

Project Structure

```

BlogApp/
|
├─ app.js                # Main application entry point
|
├─ config/
|   └─ database.js        # MongoDB connection configuration
|
├─ Module/ (Models)
|   ├── user.model.js     # User schema and model
|   └─ blog.model.js      # Blog schema and model
|
├─ controller/
|   ├── user.controller.js # User business logic
|   └─ blog.controller.js  # Blog business logic
|
├─ routers/
|   ├── user.routes.js     # User API routes
|   └─ blog.routes.js      # Blog API routes
|
├─ middleware/
|   ├── auth.middleware.js  # JWT authentication middleware
|   └─ error.middleware.js  # Global error handling middleware
|
└─ utils/
    ├── CustomError.js     # Custom error class
    ├── jwt.utils.js       # JWT token generation utility
    ├── catchAsync.util.js  # Async error handler wrapper
    └─ asyncHandler.util.js # (Empty file)
  
```

Application Flow

Startup Flow:

1. app.js loads
2. dotenv.config() loads environment variables
3. connectDB() establishes MongoDB connection
4. Express app is configured with middleware
5. Routes are registered
6. Error middleware is added (LAST)
7. Server starts listening on specified PORT

Request Flow (General):

```

Request → CORS Middleware → cookieParser → express.json()
      → Route Matching → Authentication (if required)
      → Controller → Model → Database
      → Response / Error Middleware
  
```



File-by-File Detailed Explanation

1 app.js - Application Entry Point

Purpose:

This is the heart of your application. It initializes and configures the Express server.

Line-by-Line Explanation:

```
import errorHandler from "../middleware/error.middleware.js";
import dotenv from "dotenv";
dotenv.config();
```

What happens here:

- Import error middleware (will be used at the end)
- Import `dotenv` package to read `.env` file
- `dotenv.config()` reads the `.env` file and loads all variables into `process.env`
- **MUST be called BEFORE importing other files** that use `process.env`

```
console.log(process.env);
```

- Logs all environment variables (for debugging)
- Shows: PORT, MONGODB_URL, SECRET_KEY, etc.

```
import cookieParser from "cookie-parser";
import cors from "cors";
import express from "express";
```

- **cookieParser:** Middleware to parse cookies from requests
- **cors:** Cross-Origin Resource Sharing - allows frontend to communicate with backend
- **express:** The web framework

```
import { connectDB } from "../config/database.js";
connectDB();
```

- Import database connection function
- **Call it immediately** to establish MongoDB connection

```
const app = express();
```

- Create Express application instance
- `app` is now your server object

```
app.use(
  cors({
    origin: ["http://localhost:5173"],
    credentials: true,
    methods: ["GET", "POST", "PATCH", "DELETE"],
  })
);
```

CORS Configuration Explained:

- `origin`: Only requests from `http://localhost:5173` (React dev server) are allowed
- `credentials: true`: Allows cookies to be sent between frontend and backend
- `methods`: Only these HTTP methods are permitted

Why needed?

- Browser security prevents frontend (localhost:5173) from calling backend (localhost:9000) by default
- CORS configuration gives permission

```
app.use(cookieParser());
```

- Parses cookies from incoming requests
- Makes cookies available at `req.cookies.token`

```
app.use(express.json());
```

- Parses incoming JSON data in request body
- Makes data available at `req.body`
- **Without this, req.body will be undefined**

```
app.use("/api/users", userRoutes);
app.use("/api/blogs", blogRoutes);
```

Route Registration:

- All routes in `userRoutes` will be prefixed with `/api/users`
- All routes in `blogRoutes` will be prefixed with `/api/blogs`

Example:

- If `userRoutes` has `/add`, full URL becomes `/api/users/add`

```
app.use(errorMiddleware);
```

CRITICAL: Error middleware MUST be declared LAST

- Catches all errors from previous middleware/routes
- Centralized error handling

```
app.listen(process.env.PORT, (err) => {
  if (err) console.log(err);
  console.log("server running at", process.env.PORT);
});
```

- Starts the server on specified PORT
- Callback runs after server successfully starts

2 database.js - MongoDB Connection**Purpose:**

Establishes connection to MongoDB database.

```
import mongoose from "mongoose";

export async function connectDB() {
  await mongoose.connect(process.env.MONGODB_URL);
  console.log("database connected");
}
```

What happens:

1. `mongoose.connect()` is an **async function**
2. It connects to MongoDB using connection string from `.env`
3. Connection string format: `mongodb://localhost:27017/blogAPP`
 - `localhost:27017` → MongoDB server location
 - `blogAPP` → Database name (auto-created if doesn't exist)

Important:

- Must be called in `app.js` before server starts
- Uses `await` because connection takes time
- If connection fails, app crashes (intentional - can't run without DB)

3 user.model.js - User Schema & Model

Purpose:

Defines the structure of User documents in MongoDB and adds password hashing.

Complete Breakdown:

```
import bcryptjs from "bcryptjs";
import mongoose from "mongoose";
```

- **bcryptjs:** For password hashing (encryption)
- **mongoose:** ODM (Object Data Modeling) for MongoDB

```
let userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, lowercase: true, trim: true },
    password: { type: String, required: true },
    email: {
      type: String,
      required: true,
      lowercase: true,
      trim: true,
      unique: true,
    },
  },
  { timestamps: true }
);
```

Schema Definition:

- **name:**
 - Must be a string
 - Required field
 - `lowercase: true` → Converts to lowercase automatically
 - `trim: true` → Removes whitespace from both ends
- **password:**
 - String type, required
 - Will be hashed before saving (see pre-hook below)
- **email:**
 - String, required, lowercase, trim
 - `unique: true` → No two users can have same email (creates index)
- **timestamps: true:**

- Automatically adds `createdAt` and `updatedAt` fields
- MongoDB manages these automatically

```
userSchema.pre("save", async function (next) {
  let salt = await bcryptjs.genSalt(10);
  console.log(salt);
  let hashedPassword = await bcryptjs.hash(this.password, 10);
  this.password = hashedPassword;
  next();
});
```

PRE HOOK - Critical Concept:

This is a **middleware function** that runs BEFORE saving a document.

Flow:

1. User registers with password "abc123"
2. Before saving to DB, this hook runs
3. `bcryptjs.genSalt(10)` generates a random salt
 - Salt example: `$2b$10$Zkb9RYJhIVDIHOf..an44e`
 - `$2b` → Algorithm identifier
 - `10` → Cost factor (how complex the hash is)
4. `bcryptjs.hash(this.password, 10)` creates hashed password
 - `this.password` → Original password from req.body
 - Result: Long encrypted string (e.g., `$2b$10$...64characters...`)
5. `this.password = hashedPassword` → Replace plain password with hash
6. `next()` → Continue to save the document

Why hash passwords?

- Never store plain passwords in database
- If database is compromised, passwords are safe
- One-way encryption (cannot reverse to get original password)

```
userSchema.methods.comparePassword = async function (enteredPassword) {
  return await bcryptjs.compare(enteredPassword, this.password);
};
```

Custom Instance Method:

- Adds a method to each user document
- Used during login to verify password

How it works:

1. User enters password during login: "abc123"
2. `bcryptjs.compare("abc123", hashedPasswordFromDB)`
3. bcrypt hashes the entered password with same salt
4. Compares both hashes
5. Returns `true` if match, `false` otherwise

Usage in controller:

```
let isMatch = await existingUser.comparePassword(enteredPassword);
```

```
let userModel = mongoose.model("User", userSchema);  
export default userModel;
```

Model Creation:

- `mongoose.model("User", userSchema)` creates a model
 - Model name: "User"
 - MongoDB collection name: **"users"** (lowercase + plural)
 - Export model to use in controllers
-

4 blog.model.js - Blog Schema & Model

Purpose:

Defines structure of Blog documents with reference to User.

```
import mongoose from "mongoose";

let blogSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: [true, "title is required"],
      trim: true,
      unique: true,
    },
    description: {
      type: String,
      required: true,
      minlength: [10, "at-least 10 characters are required"],
    },
    createdBy: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
  },
  { timestamps: true }
);
```

Field Breakdown:

- **title:**
 - String, required with custom error message
 - `trim: true` → Remove whitespace
 - `unique: true` → Each blog must have unique title
- **description:**
 - String, required
 - `minlength: [10, "error message"]` → Validation with custom message
 - Must be at least 10 characters
- **createdBy:**
 - **CRITICAL CONCEPT - REFERENCING**
 - Type: `ObjectId` (MongoDB's unique identifier type)
 - `ref: "User"` → References User model
 - Stores the user's `_id` who created the blog

Example:

```
{
  _id: "blog123",
  title: "My Blog",
  description: "Content...",
  createdBy: "user456" // This is the User's _id
}
```

timestamps: true:

- Auto-adds `createdAt` and `updatedAt`

```
export default mongoose.model("Blog", blogSchema);
```

- Model name: "Blog"
 - Collection name: "blogs" (lowercase + plural)
-

5 user.routes.js - User API Routes

Purpose:

Defines all user-related API endpoints.

```
import { Router } from "express";
import {
  addUser,
  deleteUser,
  getUser,
  getUsers,
  login,
  logout,
  updateUser,
} from "../controller/user.controller.js";

let router = Router();
```

Router Creation:

- `Router()` creates a mini-app for routing
- Can be mounted on main app with a prefix

Route Definitions:

```
router.post("/add", addUser);
```

- **Full URL:** `POST /api/users/add`
- **Purpose:** Register new user
- **Controller:** `addUser` function
- **Request body:** `{ name, email, password }`

```
router.get("/all", getUsers);
```

- **Full URL:** `GET /api/users/all`
- **Purpose:** Get all users
- **No request body needed**

```
router.post("/login", login);
```

- **Full URL:** `POST /api/users/login`
- **Purpose:** User login
- **Request body:** `{ email, password }`
- **Response:** Sets cookie with JWT token

```
router.post("/logout", logout);
```

- **Full URL:** `POST /api/users/logout`
- **Purpose:** User logout
- **Clears cookie**

```
router.delete("/:id", deleteUser);
```

- **Full URL:** `DELETE /api/users/:id`
- **Example:** `DELETE /api/users/123abc`
- **:id is a parameter** (dynamic value)
- Accessible in controller: `req.params.id`

```
router.patch("/:id", updateUser);
```

- **Full URL:** `PATCH /api/users/:id`
- **Example:** `PATCH /api/users/123abc`
- **Request body:** Fields to update `{ name: "New Name" }`

```
router.get("/:id", getUser);
```

- **Full URL:** `GET /api/users/:id`
- **Example:** `GET /api/users/123abc`
- Gets single user by ID

Parameter Concept:

- `:id` is a placeholder

- Actual value is captured in `req.params.id`
- Example: `/api/users/abc123` → `req.params.id = "abc123"`

```
export default router;
```

- Export router to use in app.js

6 blog.routes.js - Blog API Routes

```
import { Router } from "express";
import {
  addBlog,
  deleteBlog,
  getBlog,
  getBlogs,
  updateBlog,
} from "../controller/blog.controller.js";
import { authentication } from "../middleware/auth.middleware.js";

const router = Router();
```

Protected Routes - Authentication Middleware:

```
router.post("/add", authentication, addBlog);
```

- **Full URL:** `POST /api/blogs/add`
- **CRITICAL:** `authentication` middleware runs FIRST
- Flow: Request → authentication → addBlog
- If authentication fails, addBlog is never called

How middleware works:

1. Request comes with cookie (token)
2. `authentication` middleware verifies token
3. If valid, adds user info to `req.myUser`
4. Calls `next()` to proceed to `addBlog`
5. If invalid, returns error (addBlog not reached)

```
router.get("/all", getBlogs);
```

- **Full URL:** `GET /api/blogs/all`
- **No authentication required** (public route)

- Anyone can view all blogs

```
router.get("/one/:id", getBlog);
```

- **Full URL:** `GET /api/blogs/one/:id`
- Example: `GET /api/blogs/one/blog123`
- Gets single blog with populated user details

```
router.patch("/edit/:id", authentication, updateBlog);
```

- **Full URL:** `PATCH /api/blogs/edit/:id`
- **Protected route** (authentication required)
- Only creator can update their blog

```
router.delete("/delete/:id", deleteBlog);
```

- **Full URL:** `DELETE /api/blogs/delete/:id`
- Deletes blog by ID

7 user.controller.js - User Business Logic

This file contains all the logic for user operations.

1. addUser - User Registration

```
export const addUser = async (req, res, next) => {
  try {
    const { email, password, name } = req.body;
    let newUser = await userModel.create({
      email,
      password,
      name,
    });
    res.status(201).json({
      success: true,
      message: "User added successfully",
      newUser,
    });
  } catch (error) {
    next(error);
  }
};
```

Flow:

1. Extract data from `req.body` (sent by frontend)
2. `userModel.create()` creates new document in database
 - **PRE HOOK runs here** → Password gets hashed automatically
3. Returns 201 (Created) status with user data
4. If error occurs, pass to error middleware via `next(error)`

Important:

- Password hashing happens automatically (pre-hook in model)
- No need to manually hash here

2. getUsers - Get All Users

```
export const getUsers = async (req, res) => {
  let users = await userModel.find();
  if (users.length === 0)
    return res.status(200).json({
      success: false,
      message: "No users found",
    });
  res.status(200).json({
    success: true,
    noOfUsers: users.length,
    message: "users fetched",
    users,
  });
};
```

Flow:

1. `userModel.find()` → Returns array of all users
2. If array is empty, return message
3. Otherwise, return users with count

3. getUser - Get Single User

```
export const getUser = async (req, res) => {
  try {
    let userID = req.params.id;
    let user = await userModel.findById(userID);
    if (!user)
      return res.status(404).json({
        success: false,
        message: "no user found",
      });
    res.status(200).json({
      success: true,
      message: "user found",
      user,
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: "something went wrong",
      error,
    });
  }
};
```

Flow:

1. Get ID from URL parameters: `req.params.id`
2. `findById(userID)` searches database
3. If not found, returns `null`
4. Return 404 if null, otherwise return user

4. updateUser - Update User

```
export const updateUser = async (req, res) => {
  let userId = req.params.id;
  let updatedUser = await userModel.findByIdAndUpdate(userId, req.body, {
    new: true,
  });
  if (!updatedUser)
    return res.status(404).json({
      success: false,
      message: "user not found",
    });
  res.status(200).json({
    success: true,
    message: "updated",
    updatedUser,
  });
};
```

findByIdAndUpdate Explained:

- **First argument:** ID to find
- **Second argument:** Data to update (from req.body)

- **Third argument:** Options
 - `{ new: true }` → Returns updated document (not old one)

Example:

```
// Before update: { _id: "123", name: "John", email: "john@mail.com" }  
// req.body: { name: "Johnny" }  
// After update: { _id: "123", name: "Johnny", email: "john@mail.com" }
```

5. deleteUser - Delete User

```
export const deleteUser = async (req, res) => {  
  let userId = req.params.id;  
  let deletedUser = await userModel.findByIdAndDelete(userId);  
  if (!deletedUser)  
    return res.status(404).json({  
      success: false,  
      message: "user not found",  
    });  
  res.status(200).json({  
    success: true,  
    message: "user deleted",  
  });  
};
```

Flow:

1. Find and delete in one operation
2. Returns deleted document if found
3. Returns null if not found

6. login - User Login (MOST IMPORTANT)

```
export const login = expressAsyncHandler(async (req, res, next) => {
  const { email, password } = req.body;

  // Step 1: Check if email exists
  let existingUser = await userModel.findOne({ email });
  if (!existingUser) {
    return res.status(401).json({
      success: false,
      message: "Email not found",
    });
  }

  // Step 2: Verify password
  let isMatch = await existingUser.comparePassword(password);
  console.log(isMatch);
  if (!isMatch) {
    next(new CustomError("Password did not match", 401));
  }

  // Step 3: Generate JWT token
  let token = generateToken(existingUser._id);
  console.log(token);

  // Step 4: Send token in cookie
  res
    .cookie("token", token, {
      maxAge: 1 * 60 * 60 * 1000, // 1 hour
      httpOnly: true,
    })
    .status(200)
    .json({
      success: true,
      message: "User logged in successfully",
    });
});
```

Complete Login Flow:

Step 1: Email Verification

```
let existingUser = await userModel.findOne({ email });
```

- Searches database for user with provided email
- Returns user object if found, null otherwise
- If null, return error (user not registered)

Step 2: Password Verification

```
let isMatch = await existingUser.comparePassword(password);
```

- Calls custom method defined in user model
- bcrypt compares entered password with hashed password in DB

- Returns boolean: true if match, false otherwise

Step 3: Token Generation

```
let token = generateToken(existingUser._id);
```

- Calls utility function from jwt.utils.js
- Creates JWT with user's ID as payload
- Token example: `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...`

Step 4: Send Cookie

```
res.cookie("token", token, {
  maxAge: 1 * 60 * 60 * 1000, // 1 hour in milliseconds
  httpOnly: true,
});
```

Cookie Options:

- **maxAge:** Token expires after 1 hour
- **httpOnly: true:**
 - Cookie cannot be accessed by JavaScript (document.cookie)
 - Prevents XSS (Cross-Site Scripting) attacks
 - Only server can read/write this cookie

Why cookies instead of localStorage?

- More secure (httpOnly protection)
- Automatically sent with every request
- No need to manually attach token to requests

7. logout - User Logout

```
export const logout = expressAsyncHandler(async (req, res, next) => {
  res.clearCookie("token");
  res.status(200).json({
    success: true,
    message: "User logged out successfully",
  });
});
```

Simple:

- Removes the "token" cookie
- User is now logged out
- Next request won't have token → authentication will fail

8 blog.controller.js - Blog Business Logic

1. addBlog - Create Blog (Protected)

```
export const addBlog = asyncHandler(async (req, res) => {
  const { title, description } = req.body;
  let newBlog = await blogModel.create({
    title,
    description,
    createdBy: req.myUser._id,
  });
  res.status(201).json({
    success: true,
    message: "blog added",
    newBlog,
  });
});
```

Key Points:

1. Wrapped in asyncHandler:

- No need for try-catch
- Errors automatically passed to error middleware

2. createdBy: req.myUser._id

- `req.myUser` is set by authentication middleware
- Stores the creator's user ID
- Creates reference relationship

Flow:

Request → authentication middleware → sets req.myUser → addBlog → create blog with createdBy

2. getBlogs - Get All Blogs

```
export const getBlogs = async (req, res, next) => {
  try {
    let blogs = await blogModel.find();
    if (blogs.length === 0)
      return res.status(200).json({
        success: false,
        message: "no blogs found",
      });

    res.status(200).json({
      success: true,
      message: "all blogs fetched",
      blogs,
    });
  } catch (error) {
    next(error);
  }
};
```

Simple:

- Gets all blogs from database
- Returns array of blog objects

3. getBlog - Get Single Blog with Population

```
export const getBlog = async (req, res) => {
  let blogId = req.params.id;
  let blog = await blogModel.findById(blogId).populate({
    path: "createdBy",
    select: "name email -_id",
  });
  if (!blog)
    return res.status(404).json({
      success: false,
      message: "blog not found",
    });
  res.status(200).json({
    success: true,
    message: "blog found",
    blog,
  });
};
```

POPULATE - Critical Concept:

Without populate:

```
{
  _id: "blog123",
  title: "My Blog",
  description: "Content...",
  createdBy: "user456" // Just the ID
}
```

With populate:

```
{
  _id: "blog123",
  title: "My Blog",
  description: "Content...",
  createdBy: {
    name: "John Doe",
    email: "john@example.com"
    // _id is excluded due to -_id
  }
}
```

populate() Explained:

- **path:** Which field to populate ("createdBy")
- **select:** Which fields to show from referenced document
 - "name email" → Include name and email
 - "-_id" → Exclude _id (minus sign means exclude)

How it works:

1. Find blog by ID
2. Look at `createdBy` field (has user ID)
3. Fetch user document with that ID
4. Replace ID with actual user data
5. Return populated blog

4. updateBlog - Update Blog (Protected)

```
export const updateBlog = async (req, res, next) => {
  let updatedBlog = await blogModel.findOneAndUpdate(
    { _id: req.params.id, createdBy: req.myUser._id },
    req.body,
    { new: true }
  );

  if (!updatedBlog) next(new CustomError("Blog not found", 404));

  res.status(200).json({
    success: true,
    message: "Blog updated successfully",
    updateBlog,
  });
};
```

Critical Security Feature:

```
{ _id: req.params.id, createdBy: req.myUser._id }
```

This ensures:

- Blog ID matches (correct blog)
- createdBy matches logged-in user (authorization)
- **User can only update THEIR OWN blogs**

Example:

- User A (ID: user123) tries to update Blog B (createdBy: user456)
- Query: Find blog where `_id = blogB AND createdBy = user123`
- No match found → returns null → Error returned

This is AUTHORIZATION (not authentication):

- Authentication: "Are you logged in?"
- Authorization: "Do you have permission to do this?"

5. deleteBlog - Delete Blog

```
export const deleteBlog = async (req, res) => {};
```

Note: Implementation is empty in your code (TODO)

9 auth.middleware.js - Authentication Middleware

Purpose:

Verifies JWT token and authenticates user.

```
import jwt from "jsonwebtoken";
import userModel from "../Module/user.model.js";
import CustomError from "../utils/CustomError.js";

export const authentication = async (req, res, next) => {
  let token = req?.cookies?.token;
  if (!token) {
    next(new CustomError("Please login first", 401));
  }
  let decodedToken = jwt.verify(token, process.env.SECRET_KEY);
  let user = await userModel.findById(decodedToken.id);
  req.myUser = user;
  next();
};
```

Complete Flow Breakdown:

Step 1: Extract Token

```
let token = req?.cookies?.token;
```

- `req.cookies` → Parsed by cookie-parser middleware
- `?.` → Optional chaining (safe access, won't crash if undefined)
- Token was sent by browser (set during login)

Step 2: Check Token Exists

```
if (!token) {
  next(new CustomError("Please login first", 401));
}
```

- If no token found → User not logged in
- Pass error to error middleware
- **Controller function is NOT called**

Step 3: Verify Token

```
let decodedToken = jwt.verify(token, process.env.SECRET_KEY);
```

What jwt.verify does:

1. Takes encrypted token
2. Decrypts using SECRET_KEY
3. Returns payload (original data)

Example:

- Token: `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...`
- Decoded: `{ id: '69118557f65020643504c2fb', iat: 1762838290, exp: 1762924690 }`
 - `id` → User ID (we added this during token generation)
 - `iat` → Issued at (timestamp)
 - `exp` → Expiration time

If token is invalid or expired:

- `jwt.verify()` throws error
- Error caught by error middleware (`JsonWebTokenError`)

Step 4: Fetch User

```
let user = await userModel.findById(decodedToken.id);
req.myUser = user;
```

- Get user from database using ID from token
- **Attach user to request object**
- `req.myUser` is now available in controller

Step 5: Continue

```
next();
```

- Calls next middleware/controller
- If this was before `addBlog`, now `addBlog` runs

Why attach user to req?

- Controller needs to know WHO is making the request
- `req.myUser._id` used to set `createdBy` in blogs
- Available throughout the request lifecycle

Middleware Chain Example:

```

POST /api/blogs/add
↓
authentication middleware
  ↓ (if token valid)
  req.myUser = user object
  ↓
  next() called
  ↓
  addBlog controller
  ↓
  createdBy: req.myUser._id

```

10 error.middleware.js - Global Error Handler

Purpose:

Centralized error handling for entire application.

```

export const errorMiddleware = (err, req, res, next) => {
  let statusCode = err.statusCode || 500;
  let message = err.message || "something went wrong";

  if (err.name === "ValidationError") {
    statusCode = 400;
    message = err.message;
  }

  if (err.code === 11000) {
    statusCode = 409;
    message = "something already used";
  }

  if (err.name === "CastError") {
    statusCode = 400;
    message = "Invalid id";
  }

  if (err.name === "JsonWebTokenError") {
    statusCode = 401;
    message = "Invalid Session, Please login again";
  }

  res.status(statusCode).json({
    success: false,
    message,
    errObj: err,
  });
};

```

How It Works:

Function Signature:

```
err, req, res, next;
```

- **4 parameters** → Express recognizes this as error middleware
- `err` → Error object passed via `next(error)`

Default Values:

```
let statusCode = err.statusCode || 500;
let message = err.message || "something went wrong";
```

- Use error's statusCode/message if present
- Otherwise use defaults

Error Type Handling:

1. ValidationError (Mongoose)

```
if (err.name === "ValidationError") {
  statusCode = 400;
  message = err.message;
}
```

When it occurs:

- Required field missing
- Type mismatch (string instead of number)
- Validation rules violated (minlength, etc.)

Example:

```
// Missing required field
await userModel.create({ name: "John" }); // No email
// Error: ValidationError: email is required
```

2. Duplicate Key Error (MongoDB)

```
if (err.code === 11000) {
  statusCode = 409;
  message = "something already used";
}
```

When it occurs:

- Unique constraint violation
- Trying to insert duplicate value in unique field

Example:

```
// Email already exists in database
await userModel.create({
  email: "existing@mail.com", // Already used
  password: "123",
  name: "John",
});
// Error code: 11000 (MongoDB duplicate key error)
```

3. CastError (Mongoose)

```
if (err.name === "CastError") {
  statusCode = 400;
  message = "Invalid id";
}
```

When it occurs:

- Invalid ObjectId format
- Wrong data type

Example:

```
// Invalid ID format
await userModel.findById("123"); // Not a valid MongoDB ObjectId
// Valid ObjectId: "507f1f77bcf86cd799439011" (24 hex characters)
// Error: CastError
```

4. JsonWebTokenError (JWT)

```
if (err.name === "JsonWebTokenError") {
  statusCode = 401;
  message = "Invalid Session, Please login again";
}
```

When it occurs:

- Invalid JWT token
- Tampered token
- Token signed with different secret

Example:

```
// Token verification fails
jwt.verify("invalid-token", SECRET_KEY);
// Error: JsonWebTokenError
```

Response:

```
res.status(statusCode).json({
  success: false,
  message,
  errObj: err,
});
```

- Sends consistent error response
- `errObj` contains full error details (for debugging)

Why Use Global Error Middleware?

- **DRY Principle:** Don't repeat error handling in every controller
 - **Consistency:** All errors formatted the same way
 - **Centralized:** Easy to modify error responses
-

1 1 CustomError.js - Custom Error Class**Purpose:**

Create custom error objects with statusCode.

```
class CustomError extends Error {
  constructor(message, statusCode) {
    super();
    this.message = message;
    this.statusCode = statusCode;
  }
}
export default CustomError;
```

Class Inheritance Explained:

```
class CustomError extends Error
```

- `extends Error` → Inherits from JavaScript's built-in Error class
- Gets all Error properties/methods (name, stack trace, etc.)

Constructor:

```
constructor(message, statusCode) {
  super(); // Call parent class constructor
  this.message = message;
  this.statusCode = statusCode;
}
```

super() Explanation:

- **MUST be first statement** in constructor when extending
- Calls parent (Error) class constructor
- Initializes Error properties

Usage Example:

```
// In controller
if (!user) {
  throw new CustomError("User not found", 404);
}

// Or with next()
next(new CustomError("Unauthorized", 401));
```

What happens:

1. Create CustomError object
2. Error has both `message` and `statusCode`
3. Passed to error middleware via `next()`
4. Error middleware extracts `statusCode` and `message`

Built-in Error vs CustomError:

Built-in Error:

```
throw new Error("Something went wrong");
// Has: message
// No statusCode → defaults to 500
```

CustomError:

```
throw new CustomError("Not found", 404);
// Has: message AND statusCode
// Error middleware uses correct status code
```

1 2 jwt.utils.js - JWT Token Generation

Purpose:

Generate JWT tokens for authentication.

Syntax:

```
jwt.sign(payload, secretKey, options);
```

1. **Payload** - { id }

- ## 2. Secret Key - `process.env.SECRET_KEY`

- ### 3. Options - { expiresIn: "1d" }

- eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6InV0ayIsImhhbmI6Imt2MDY4MDk2MiwiZXhwIjoxNzYwNzY3MzYyYyYyLjg27vk8jjZ2qgIUIiOiJ10QscLR9WUmg1quctGRtpzjMeg

```
{ "alg": "HS256", "typ": "JWT" }
```

2. Payload (Your data)

```
{ "id": "utk", "iat": 1760680962, "exp": 1760767362 }
```

- `id`: What we passed
- `iat`: Issued At timestamp
- `exp`: Expiration timestamp

3. Signature (Verification)

- Created using: `HMACSHA256(header + payload, SECRET_KEY)`
- Ensures token wasn't tampered with

Encryption:

- Token is **encoded** (not encrypted)
- Anyone can decode and read payload
- But cannot modify without SECRET_KEY
- Signature verification ensures authenticity

Security:

- If someone changes payload, signature won't match
- `jwt.verify()` will throw error
- **Never store passwords in token**

Usage in Application:

```
// During login
let token = generateToken(user._id);
// Returns: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

// Send to client
res.cookie("token", token);

// Client stores it
// Sends with every request

// Server verifies
let decoded = jwt.verify(token, SECRET_KEY);
// Returns: { id: "user123", iat: ..., exp: ... }
```

1 3 catchAsync.util.js - Async Error Handler

Purpose:

Wrapper function to handle async errors automatically.

```
export function asyncHandler(fn) {
  return function (req, res, next) {
    Promise.resolve(fn(req, res, next)).catch(next);
  };
}
```

What This Does:

Without asyncHandler:

```
export const addBlog = async (req, res, next) => {
  try {
    // ... code
  } catch (error) {
    next(error); // Must manually catch and pass to next
  }
};
```

With asyncHandler:

```
export const addBlog = asyncHandler(async (req, res, next) => {
  // ... code
  // No try-catch needed!
});
```

How It Works - Step by Step:

1. Function Wrapper Concept:

```
asyncHandler(myFunction);
```

- Takes a function as input
- Returns a new wrapped function
- Wrapped function has error handling built-in

2. Return Function:

```
return function (req, res, next) {
  // This function is called when route is hit
};
```

- Returns a new function
- This function accepts Express parameters (req, res, next)
- Express calls this function when route matches

3. Promise.resolve:

```
Promise.resolve(fn(req, res, next));
```

- `fn` is your async controller function
- Executes the controller
- Wraps result in Promise (even if already a promise)
- Ensures consistent promise handling

4. Catch Errors:

```
.catch(next)
```

- If any error occurs in `fn`
- Catch it automatically
- Pass to `next()` → Error middleware

Visual Flow:

```

Request comes in
↓
asyncHandler wrapper function called
↓
Original controller (fn) executed
↓
If error occurs → .catch(next) → Error middleware
↓
If success → Response sent

```

Example Comparison:

WITHOUT asyncHandler:

```

export const getUser = async (req, res, next) => {
  try {
    let user = await userModel.findById(req.params.id);
    if (!user) {
      return res.status(404).json({ message: "Not found" });
    }
    res.json({ user });
  } catch (error) {
    next(error);
  }
};

```

WITH asyncHandler:

```
export const getUser = asyncHandler(async (req, res) => {
  let user = await userModel.findById(req.params.id);
  if (!user) {
    return res.status(404).json({ message: "Not found" });
  }
  res.json({ user });
});
```

Benefits:

- ✓ Cleaner code (no try-catch boilerplate)
- ✓ Automatic error handling
- ✓ All async errors caught
- ✓ Consistent error flow

Alternative Package: Your code also uses `express-async-handler` package which does the same thing.

Complete Request Flow Examples

Example 1: User Registration Flow

Client Request:

```
POST /api/users/add
Body: {
  "name": "John Doe",
  "email": "john@example.com",
  "password": "mypassword123"
}
```

Complete Flow:

1. Request hits Express server
↓
2. CORS middleware
 - Checks if origin is allowed (localhost:5173)
 - Adds CORS headers
 ↓
3. cookieParser middleware
 - Parses cookies (none in registration)
 ↓
4. express.json() middleware
 - Parses JSON body
 - req.body = { name, email, password }
 ↓
5. Route matching: app.use("/api/users", userRoutes)
 - Matches /api/users prefix
 - Forwards to userRoutes
 ↓
6. Route matching: router.post("/add", addUser)
 - Matches /add path
 - Calls addUser controller
 ↓
7. addUser controller:
 - Extracts data from req.body
 - Calls userModel.create()
 ↓
8. Mongoose pre-hook triggers:
 - Generates salt
 - Hashes password
 - Replaces plain password with hash
 ↓
9. MongoDB saves document:
 - New user created in 'users' collection
 ↓
10. Response sent to client:
 - Status: 201 Created
 - Body: { success: true, message: "User added successfully", newUser }

If Error Occurs:

```

Any step throws error
↓
Caught by try-catch or asyncHandler
↓
next(error) called
↓
errorMiddleware processes error
↓
Sends error response

```

Example 2: User Login Flow

Client Request:

```

POST /api/users/login
Body: {
  "email": "john@example.com",
  "password": "mypassword123"
}

```

Complete Flow:

```

1. Request → CORS → cookieParser → express.json()
   ↓
2. Route: POST /api/users/login → login controller
   ↓
3. login controller:
   a) Check email exists:
      await userModel.findOne({ email: "john@example.com" })
      ↓
      If not found → Return 401 error
      If found → Continue

   b) Verify password:
      await existingUser.comparePassword("mypassword123")
      ↓
      bcrypt.compare(entered, hashedInDB)
      ↓
      Returns true/false
      ↓
      If false → Return 401 error
      If true → Continue

   c) Generate JWT token:
      generateToken(existingUser._id)
      ↓
      jwt.sign({ id: "user123" }, SECRET_KEY, { expiresIn: "1d" })
      ↓
      Returns: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

   d) Send token in cookie:
      res.cookie("token", token, { maxAge: 3600000, httpOnly: true })
      ↓
      Cookie set in response headers
      ↓
      Browser stores cookie

   e) Send success response:
      Status: 200
      Body: { success: true, message: "User logged in successfully" }

```

What Client Receives:

```

Response Headers:
  Set-Cookie: token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...; Max-Age=3600; HttpOnly

Response Body:
  { success: true, message: "User logged in successfully" }

```

Example 3: Creating a Blog (Protected Route)

Client Request:

```
POST /api/blogs/add
Headers: {
  Cookie: "token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
Body: {
  "title": "My First Blog",
  "description": "This is an amazing blog post about Node.js"
}
```

Complete Flow:

1. Request → CORS → cookieParser → express.json()
`req.cookies = { token: "eyJhbG..." }`
`req.body = { title: "My First Blog", description: "..." }`
↓
2. Route: POST /api/blogs/add
`router.post("/add", authentication, addBlog)`
↓
3. authentication middleware RUNS FIRST:
 - a) Extract token:
`let token = req.cookies.token`
↓
If no token → `next(CustomError("Please login first", 401))`
→ Error middleware → Response sent → STOP
↓
Token found → Continue
 - b) Verify token:
`jwt.verify(token, SECRET_KEY)`
↓
Decrypts token
↓
Returns: `{ id: "user123", iat: 1234567890, exp: 1234654290 }`
↓
If invalid/expired → Throws `JsonWebTokenError`
→ Error middleware → Response sent → STOP
↓
Valid → Continue
 - c) Fetch user:
`await userModel.findById(decodedToken.id)`
↓
Returns user object
↓
`req.myUser = user // Attach to request`
 - d) Call `next()`:
`next()`
↓
Proceeds to `addBlog` controller
↓
4. `addBlog` controller:
 - a) Extract data:
`const { title, description } = req.body`
 - b) Create blog:
`await blogModel.create({`
`title: "My First Blog",`
`description: "This is an amazing...",`
`createdBy: req.myUser._id // From authentication middleware!`
`})`
↓
MongoDB saves to 'blogs' collection
↓

```
{
  _id: "blog123",
  title: "My First Blog",
  description: "This is an amazing...",
  createdBy: "user123", // References User document
  createdAt: "2025-01-15T10:30:00.000Z",
  updatedAt: "2025-01-15T10:30:00.000Z"
}
```

```
c) Send response:  
Status: 201 Created  
Body: { success: true, message: "blog added", newBlog: {...} }
```

Key Points:

- Authentication middleware runs BEFORE controller
 - If authentication fails, controller never executes
 - `req.myUser` carries user info from middleware to controller
 - Blog is linked to user via `createdBy` field
-

Example 4: Getting Single Blog with User Details**Client Request:**

```
GET / api / blogs / one / blog123;
```

Complete Flow:

1. Request → Middleware chain → Route matching

↓

2. Route: GET /api/blogs/one/:id → getBlog controller
req.params.id = "blog123"

↓

3. getBlog controller:

a) Extract ID:

```
let blogId = req.params.id // "blog123"
```

b) Find and populate:

```
await blogModel.findById("blog123").populate({
  path: "createdBy",
  select: "name email -_id"
})
```

↓

Step-by-step populate:

i) Find blog document:

```
{
  _id: "blog123",
  title: "My First Blog",
  description: "Content...",
  createdBy: "user123" // Just ID initially
}
```

ii) Look at 'createdBy' field → "user123"

iii) Fetch user with ID "user123":

```
{
  _id: "user123",
  name: "John Doe",
  email: "john@example.com",
  password: "hashed..."
}
```

iv) Select only: name, email (exclude _id)

v) Replace ID with user data:

```
{
  _id: "blog123",
  title: "My First Blog",
  description: "Content...",
  createdBy: {
    name: "John Doe",
    email: "john@example.com"
  }
}
```

c) Send response:

Status: 200

Body: { success: true, message: "blog found", blog: {...} }

Response Received by Client:

```
{
  "success": true,
  "message": "blog found",
  "blog": {
    "_id": "blog123",
    "title": "My First Blog",
    "description": "This is an amazing blog post about Node.js",
    "createdBy": {
      "name": "John Doe",
      "email": "john@example.com"
    },
    "createdAt": "2025-01-15T10:30:00.000Z",
    "updatedAt": "2025-01-15T10:30:00.000Z"
  }
}
```

Example 5: Updating Blog (Authorization Check)

Client Request:

```
PATCH /api/blogs/edit/blog123
Headers: {
  Cookie: "token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
Body: {
  "title": "Updated Blog Title"
}
```

Complete Flow:

1. Request → Middleware → Route matching
↓
2. Route: PATCH /api/blogs/edit/:id
router.patch("/edit/:id", authentication, updateBlog)
↓
3. authentication middleware:
 - Verifies token
 - Sets req.myUser = logged-in user
 - req.myUser._id = "user123"
 ↓
4. updateBlog controller:
 - a) Find and update with conditions:


```
await blogModel.findOneAndUpdate(
  {
    _id: req.params.id,          // "blog123"
    createdBy: req.myUser._id    // "user123"
  },
  req.body,                    // { title: "Updated Blog Title" }
  { new: true }
)
```

 ↓

MongoDB query:
"Find blog where _id = 'blog123' AND createdBy = 'user123'"

 ↓

Scenario 1: User IS the creator

 - Blog found
 - Update applied
 - Returns updated document
 ↓

Scenario 2: User is NOT the creator

 - Blog with _id exists but createdBy doesn't match
 - No document found
 - Returns null
 ↓
 - b) Check result:


```
if (!updatedBlog)
  → next(CustomError("Blog not found", 404))
  → Can't update someone else's blog!

else
  → Send success response
```

Authorization Flow Diagram:

User A (ID: user123) tries to update Blog X (createdBy: user123)

↓

Query: { _id: "blogX", createdBy: "user123" }

↓

✅ Match found → Update allowed

User A (ID: user123) tries to update Blog Y (createdBy: user456)

↓

Query: { _id: "blogY", createdBy: "user123" }

↓

❌ No match → Update denied → 404 error

This is the difference between:

- **Authentication:** "Who are you?" (checked by auth middleware)
- **Authorization:** "Can you do this?" (checked in controller)

🔑 Key Concepts Explained

1. Middleware in Express

What is Middleware? Functions that have access to `req`, `res`, and `next` in the request-response cycle.

Types:

a) Application-level middleware:

```
app.use(express.json()); // Runs for ALL routes
app.use(cookieParser());
```

b) Router-level middleware:

```
router.post("/add", authentication, addBlog);
//                ^^^^^^^^^^^^^^^^^^^^^^ <- Middleware for this route only
```

c) Error-handling middleware:

```
app.use((err, req, res, next) => {
  // 4 parameters → Express knows it's error middleware
});
```

Middleware Chain:

```
app.use(middleware1);
app.use(middleware2);
app.use("/route", middleware3, controller);
```

Flow:

```
Request → middleware1 → next() → middleware2 → next()
      → middleware3 → next() → controller → Response
```

Each middleware must:

- Call `next()` to proceed, OR
- Send a response, OR
- Pass error to `next(error)`

2. Async/Await in Node.js

Synchronous vs Asynchronous:

Synchronous (Blocking):

```
let result = someFunction(); // Wait for completion
console.log(result); // Runs after above finishes
```

Asynchronous (Non-blocking):

```
someFunction().then((result) => {
  console.log(result);
});
// Code here runs immediately, doesn't wait
```

Async/Await (Clean async code):

```
async function myFunction() {
  let result = await someAsyncFunction();
  console.log(result); // Waits for result
}
```

Rules:

- `await` can only be used inside `async` function
- `await` pauses execution until Promise resolves
- Must use try-catch for error handling (or use `asyncHandler`)

Example in BlogApp:

```
export const addUser = async (req, res, next) => {
  try {
    let newUser = await userModel.create(req.body);
    //      ^^^^^ Wait for database operation
    res.json({ newUser });
  } catch (error) {
    next(error);
  }
};
```

3. MongoDB Relationships

Two ways to relate documents:

a) Embedding (Nested documents):

```
{
  _id: "blog123",
  title: "My Blog",
  author: {
    name: "John",
    email: "john@example.com"
  }
}
```

Pros: Single query, fast **Cons:** Data duplication, hard to update

b) Referencing (Used in BlogApp):

```
// Blog document
{
  _id: "blog123",
  title: "My Blog",
  createdBy: "user123" // Reference to User
}

// User document
{
  _id: "user123",
  name: "John",
  email: "john@example.com"
}
```

Pros: No duplication, easy updates **Cons:** Need populate() for related data

Populate Example:

```
// Without populate
let blog = await blogModel.findById("blog123");
// blog.createdBy = "user123" (just ID)

// With populate
let blog = await blogModel.findById("blog123").populate("createdBy");
// blog.createdBy = { _id: "user123", name: "John", ... } (full object)
```

4. HTTP Status Codes

Used in BlogApp:

- **200 OK:** Successful GET, PATCH, DELETE
- **201 Created:** Successful POST (resource created)
- **400 Bad Request:** Validation errors, invalid data
- **401 Unauthorized:** Not logged in, invalid credentials
- **404 Not Found:** Resource doesn't exist
- **409 Conflict:** Duplicate entry (email already exists)
- **500 Internal Server Error:** Unexpected server error

Usage:

```
res.status(201).json({ ... }); // Created
res.status(404).json({ ... }); // Not Found
```

5. Environment Variables (.env)

Why use .env?

- Keep secrets out of code
- Different configs for dev/prod
- Never commit .env to Git

Example .env file:

```
PORT=9000
MONGODB_URL=mongodb://localhost:27017/blogAPP
SECRET_KEY=my-super-secret-jwt-key-12345
```

Access in code:

```
dotenv.config(); // Load .env
console.log(process.env.PORT); // "9000"
```

Important:

- `dotenv.config()` must be called FIRST
 - Add `.env` to `.gitignore`
 - Use `.env.example` for template (without actual values)
-

6. CORS (Cross-Origin Resource Sharing)

The Problem:

```
Frontend: http://localhost:5173  
Backend: http://localhost:9000
```

Browser blocks requests between different origins (security).

The Solution:

```
app.use(  
  cors({  
    origin: ["http://localhost:5173"], // Allow this origin  
    credentials: true, // Allow cookies  
  })  
);
```

What happens:

1. Frontend makes request to backend
2. Browser checks CORS headers
3. Backend sends: `Access-Control-Allow-Origin: http://localhost:5173`
4. Browser allows the request

Without CORS:

✗ CORS Error: Request blocked

With CORS:

✓ Request allowed

7. Cookies vs LocalStorage

Cookies:

```
res.cookie("token", "value", { httpOnly: true });
```

- Automatically sent with every request
- Can be httpOnly (JS can't access)
- Has expiration
- More secure for auth tokens

LocalStorage:

```
localStorage.setItem("token", "value");
```

- Manual management (attach to requests)
- Accessible by JavaScript
- Vulnerable to XSS attacks
- No expiration

BlogApp uses Cookies because:

- More secure (httpOnly)
- Automatic (no manual header management)
- Better for authentication

8. Password Hashing with bcrypt

Why hash?

- Never store plain passwords
- One-way encryption
- Even with same password, hash is different (due to salt)

Salt:

```
let salt = await bcryptjs.genSalt(10);  
// Result: $2b$10$Zkb9RYJhIVDIH0f..an44e
```

- Random string
- Makes each hash unique
- Prevents rainbow table attacks

Hashing:

```
let hash = await bcryptjs.hash("password123", salt);
// Result: $2b$10$...60+ characters...
```

Comparing:

```
let isMatch = await bcryptjs.compare("password123", hash);
// Returns: true or false
```

Why it's secure:

- Cannot reverse hash to get password
- Each hash is unique (even for same password)
- Computationally expensive (slow brute force)

9. JWT (JSON Web Tokens)

Structure:

```
header.payload.signature
```

Example:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6InVzZXIiLCJpYXQiOi0jE3NjA2ODA5NjJ9.g27vk8jJz2qgIUIiJ10QscLR9WUmg1quctGRtpzjMeg
```

Parts:

1. Header (Algorithm & Type):

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

2. Payload (Data):

```
{
  "id": "user123",
  "iat": 1760680962, // Issued at
  "exp": 1760767362 // Expires at
}
```

3. Signature (Verification):

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  SECRET_KEY
)
```

How JWT Authentication Works:

1. User logs in
↓
2. Server verifies credentials
↓
3. Server generates JWT with user ID
 `jwt.sign({ id: user._id }, SECRET_KEY)`
↓
4. Server sends JWT to client (in cookie)
 `res.cookie("token", jwt)`
↓
5. Client stores JWT (browser stores cookie)
↓
6. Client sends JWT with every request (automatic with cookies)
↓
7. Server verifies JWT
 `jwt.verify(token, SECRET_KEY)`
↓
8. If valid → Allow access
 If invalid → Deny access

JWT vs Session:

JWT (Stateless):

- Token stored on client
- Server doesn't store anything
- Scalable (no server memory needed)
- Used in BlogApp

Session (Stateful):

- Session data stored on server
- Session ID sent to client
- Server must remember sessions
- Uses more server resources

10. Mongoose Schema Types and Validations

Common Types:

```
{
  name: String,
  age: Number,
  isActive: Boolean,
  birthDate: Date,
  hobbies: [String], // Array of strings
  userId: mongoose.Schema.Types.ObjectId // Reference
}
```

Validations:

Required:

```
name: {
  type: String,
  required: true // Simple
}
// OR
email: {
  type: String,
  required: [true, "Email is required"] // With custom message
}
```

Unique:

```
email: {
  type: String,
  unique: true // Creates index, prevents duplicates
}
```

String Validations:

```
description: {
  type: String,
  minlength: [10, "Must be at least 10 characters"],
  maxlength: [500, "Cannot exceed 500 characters"],
  trim: true, // Remove whitespace
  lowercase: true, // Convert to lowercase
  uppercase: true // Convert to uppercase
}
```

Number Validations:

```
age: {
  type: Number,
  min: [18, "Must be at least 18"],
  max: [100, "Cannot exceed 100"]
}
```

Enum (Fixed values):

```
role: {
  type: String,
  enum: ["user", "admin", "moderator"],
  default: "user"
}
```

Custom Validation:

```
email: {
  type: String,
  validate: {
    validator: function(v) {
      return /^[^w-\.] + @([w-] + \. ) + [w-] {2,4} $/.test(v);
    },
    message: "Invalid email format"
  }
}
```

11. Mongoose Middleware (Hooks)

Types of Hooks:

a) Pre Hook (Before operation):

```
userSchema.pre("save", async function (next) {
  // Runs BEFORE saving document
  this.password = await bcrypt.hash(this.password, 10);
  next();
});
```

b) Post Hook (After operation):

```
userSchema.post("save", function (doc, next) {
  // Runs AFTER saving document
  console.log("User saved:", doc._id);
  next();
});
```

Operations with hooks:

- `save` - Document is saved
- `validate` - Document is validated
- `remove` - Document is removed
- `updateOne` - Document is updated
- `find` - Documents are queried

Real-world use cases:

1. Password hashing (BlogApp uses this):

```
userSchema.pre("save", async function (next) {
  if (!this.isModified("password")) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});
```

2. Auto-populate fields:

```
blogSchema.pre("find", function (next) {
  this.populate("createdBy");
  next();
});
```

3. Logging:

```
userSchema.post("remove", function (doc) {
  console.log(`User ${doc.email} was deleted`);
});
```

12. Error Handling Patterns

Pattern 1: Try-Catch (Manual)

```
export const getUser = async (req, res) => {
  try {
    let user = await userModel.findById(req.params.id);
    res.json({ user });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

Pros: Full control **Cons:** Repetitive, boilerplate code

Pattern 2: AsyncHandler (BlogApp uses this)

```
export const getUser = asyncHandler(async (req, res) => {
  let user = await userModel.findById(req.params.id);
  res.json({ user });
});
```

Pros: Clean, no try-catch needed **Cons:** Less control over error handling

Pattern 3: Global Error Middleware (BlogApp uses this)

```
app.use((err, req, res, next) => {
  let statusCode = err.statusCode || 500;
  res.status(statusCode).json({ error: err.message });
});
```

Pros: Centralized, consistent errors **Cons:** Need to pass errors correctly

Pattern 4: Custom Errors (BlogApp uses this)

```
if (!user) {
  throw new CustomError("User not found", 404);
}
```

Pros: Clean, with status codes **Cons:** Need custom class

Best Practice (Combine all):

```
// 1. AsyncHandler wrapper
export const getUser = asyncHandler(async (req, res, next) => {
  let user = await userModel.findById(req.params.id);

  // 2. Custom error with status code
  if (!user) {
    return next(new CustomError("User not found", 404));
  }

  res.json({ user });
});

// 3. Global error middleware catches everything
app.use(errorMiddleware);
```

13. Express Router Explained

Why use Router?

- Organize routes into modules
- Keep code clean and maintainable
- Reusable route groups

Without Router (Messy):

```
// app.js
app.post("/api/users/add", addUser);
app.get("/api/users/all", getUsers);
app.post("/api/blogs/add", addBlog);
app.get("/api/blogs/all", getBlogs);
// ... 50+ routes in one file
```

With Router (Clean):

```
// user.routes.js
const router = Router();
router.post("/add", addUser);
router.get("/all", getUsers);
export default router;

// blog.routes.js
const router = Router();
router.post("/add", addBlog);
router.get("/all", getBlogs);
export default router;

// app.js
app.use("/api/users", userRoutes);
app.use("/api/blogs", blogRoutes);
```

Route Composition:

```
app.use("/api/users", userRoutes)
      +
router.post("/add", addUser)
      =
Final route: POST /api/users/add
```

Nested Routers:

```
// Advanced pattern (not in BlogApp)
const adminRouter = Router();
const userRouter = Router();

adminRouter.use("/users", userRouter);
app.use("/api/admin", adminRouter);

// Result: /api/admin/users/add
```

14. RESTful API Design (Used in BlogApp)

REST Principles:

1. Resource-based URLs:

```
GET    /api/blogs    // Get all blogs
POST   /api/blogs    // Create blog
GET    /api/blogs/:id // Get one blog
PATCH /api/blogs/:id // Update blog
DELETE /api/blogs/:id // Delete blog
```

2. HTTP Methods (CRUD):

- **POST** → Create
- **GET** → Read

- **PATCH/PUT** → Update
- **DELETE** → Delete

3. Status Codes:

- 2xx → Success
- 4xx → Client error
- 5xx → Server error

4. Stateless:

- Each request contains all needed info
- Server doesn't remember previous requests
- Uses JWT for authentication

BlogApp Routes:

User Routes:

```
POST /api/users/add // Register
POST /api/users/login // Login
POST /api/users/logout // Logout
GET /api/users/all // Get all users
GET /api/users/:id // Get one user
PATCH /api/users/:id // Update user
DELETE /api/users/:id // Delete user
```

Blog Routes:

```
POST /api/blogs/add // Create blog (protected)
GET /api/blogs/all // Get all blogs
GET /api/blogs/one/:id // Get one blog
PATCH /api/blogs/edit/:id // Update blog (protected)
DELETE /api/blogs/delete/:id // Delete blog
```

15. Database Indexing

What is an Index? Like a book index - helps find data quickly.

Automatic Indexes in BlogApp:

1. `_id` field:

- MongoDB creates automatically
- Unique index
- Fast lookups by ID

2. `unique: true` fields:

```
email: {  
  type: String,  
  unique: true // Creates index  
}
```

- Email index created automatically
- Fast lookups by email
- Prevents duplicates

How Indexes Help:

Without Index:

```
// Scans ALL documents  
await userModel.findOne({ email: "john@example.com" });  
// Time: O(n) - checks every user
```

With Index:

```
// Uses index for instant lookup  
await userModel.findOne({ email: "john@example.com" });  
// Time: O(log n) - binary search
```

Trade-offs:

-  Faster reads
 -  Slower writes (must update index)
 -  Uses more disk space
-

Complete Application Lifecycle

1. Application Startup

```
// app.js execution order
```

1. Import statements (top to bottom)
 - `errorMiddleware` imported
 - `dotenv` imported
2. `dotenv.config()` executed
 - Reads `.env` file
 - Loads variables into `process.env`
3. `console.log(process.env)`
 - Logs all environment variables
4. More imports (`cookieParser`, `cors`, `express`, etc.)
5. `connectDB()` called
 - Establishes MongoDB connection
 - Waits **for** connection to complete
6. Express app created
`const app = express()`
7. Middleware registered (order matters!)
 - `app.use(cors(...))` // First
 - `app.use(cookieParser())` // Second
 - `app.use(express.json())` // Third
8. Routes registered
 - `app.use("/api/users", userRoutes)`
 - `app.use("/api/blogs", blogRoutes)`
9. **Error** middleware registered (MUST BE LAST)
 - `app.use(errorMiddleware)`
10. Server starts listening
 - `app.listen(PORT)`
 - Server ready to accept requests

2. Request Lifecycle

Example: Creating a blog post

Step 1: Client sends request
POST /api/blogs/add
Headers: { Cookie: "token=..." }
Body: { title: "...", description: "..." }
↓

Step 2: Request enters Express
↓

Step 3: CORS middleware
- Checks origin
- Adds CORS headers
- Calls next()
↓

Step 4: cookieParser middleware
- Parses cookies
- Sets req.cookies = { token: "..." }
- Calls next()
↓

Step 5: express.json() middleware
- Parses JSON body
- Sets req.body = { title: "...", description: "..." }
- Calls next()
↓

Step 6: Route matching
- Matches /api/blogs prefix
- Forwards to blogRoutes
- Matches /add path
↓

Step 7: authentication middleware
a) Extracts token from req.cookies
b) Verifies token with jwt.verify()
c) Fetches user from database
d) Sets req.myUser = user
e) Calls next()
↓

Step 8: addBlog controller
a) Extracts data from req.body
b) Creates blog with createdBy = req.myUser._id
c) Saves to MongoDB
d) Sends response
↓

Step 9: Response sent to client
Status: 201
Body: { success: true, ... }
↓

Step 10: Connection closed

If error occurs at any step:

```
Error thrown
↓
Caught by try-catch or asyncHandler
↓
next(error) called
↓
Error middleware (errorMiddleware)
↓
Error response sent
↓
Connection closed
```

Common MongoDB Operations in BlogApp

1. Create Operations

Create one document:

```
let newUser = await userModel.create({
  name: "John",
  email: "john@example.com",
  password: "hashed...",
});

// Result: Document object with _id
```

Create multiple documents:

```
let users = await userModel.insertMany([
  { name: "John", email: "john@mail.com" },
  { name: "Jane", email: "jane@mail.com" },
]);
```

2. Read Operations

Find all:

```
let users = await userModel.find();
// Returns: Array of all user documents
```

Find with conditions:

```
let activeUsers = await userModel.find({ isActive: true });
```

Find one:

```
let user = await userModel.findOne({ email: "john@example.com" });
// Returns: Single document or null
```

Find by ID:

```
let user = await userModel.findById("507f1f77bcf86cd799439011");
// Returns: Single document or null
```

Find with select (specific fields):

```
let user = await userModel.findById(id).select("name email -_id");
// Returns: { name: "John", email: "john@example.com" }
// "-_id" excludes _id field
```

Find with populate:

```
let blog = await blogModel.findById(id).populate("createdBy");
// Returns blog with full user object instead of just ID
```

Find with multiple conditions:

```
let blogs = await blogModel.find({
  createdBy: userId,
  title: { $regex: "Node", $options: "i" }, // Case-insensitive search
});
```

3. Update Operations

Update one (returns info):

```
let result = await userModel.updateOne(
  { _id: userId }, // Filter
  { $set: { name: "John" } } // Update
);
// Returns: { acknowledged: true, modifiedCount: 1, ... }
```

Find and update (returns document):

```
let updatedUser = await userModel.findByIdAndUpdate(
  userId,
  { name: "John" },
  { new: true } // Return updated document
);
// Returns: Updated document object
```

Update many:

```
let result = await userModel.updateMany(
  { isActive: false },
  { $set: { status: "inactive" } }
);
```

4. Delete Operations

Delete one:

```
let result = await userModel.deleteOne({ _id: userId });
// Returns: { acknowledged: true, deletedCount: 1 }
```

Find and delete:

```
let deletedUser = await userModel.findByIdAndDelete(userId);
// Returns: Deleted document or null
```

Delete many:

```
let result = await userModel.deleteMany({ isActive: false });
```

5. Query Operators

Comparison:

```
// Greater than
{
  age: {
    $gt: 18;
  }
}

// Less than or equal
{
  age: {
    $lte: 30;
  }
}

// Not equal
{
  status: {
    $ne: "inactive";
  }
}

// In array
{
  role: {
    $in: ["admin", "moderator"];
  }
}
```

Logical:

```
// AND (implicit)
{ name: "John", age: 25 }

// OR
{ $or: [{ name: "John" }, { name: "Jane" }] }

// NOT
{ age: { $not: { $lt: 18 } } }
```

Element:

```
// Field exists
{
  email: {
    $exists: true;
  }
}

// Type checking
{
  age: {
    $type: "number";
  }
}
```

Security Best Practices (Used in BlogApp)

1. Password Security

- ✓ **Hash passwords** (never store plain text)

```
// BlogApp does this
let hashedPassword = await bcrypt.hash(password, 10);
```

- ✓ **Use salt** (makes each hash unique)

```
let salt = await bcrypt.genSalt(10);
```

- ✓ **Pre-hook hashing** (automatic)

```
userSchema.pre("save", async function (next) {
  this.password = await bcrypt.hash(this.password, 10);
  next();
});
```

2. Authentication Security

- ✓ **Use JWT tokens**

```
let token = jwt.sign({ id: user._id }, SECRET_KEY, { expiresIn: "1d" });
```

- ✓ **httpOnly cookies** (prevent XSS)

```
res.cookie("token", token, {
  httpOnly: true, // JavaScript can't access
  maxAge: 3600000,
});
```

- ✓ **Token expiration**

```
{
  expiresIn: "1d";
} // Token expires after 1 day
```

- ✓ **Verify tokens on protected routes**

```
router.post("/add", authentication, addBlog);
```

3. Input Validation

✓ Mongoose validations

```
email: {  
  type: String,  
  required: [true, "Email is required"],  
  unique: true  
}
```

✓ Check for existence before operations

```
if (!user) {  
  return res.status(404).json({ message: "User not found" });  
}
```

4. Authorization

✓ Check ownership before updates

```
await blogModel.findOneAndUpdate(  
  { _id: blogId, createdBy: req.myUser._id }, // Must be owner  
  updateData  
);
```

5. Error Handling

✓ Don't expose sensitive errors

```
// Bad  
res.json({ error: err }); // Exposes stack trace  
  
// Good  
res.json({ message: "Something went wrong" });
```

✓ Use global error middleware

```
app.use(errorMiddleware);
```

6. Environment Variables

✓ Keep secrets in .env

```
SECRET_KEY=my-secret-key  
MONGODB_URL=mongodb://localhost:27017/blogAPP
```

✓ Never commit .env to Git

```
# .gitignore  
.env
```

7. CORS Configuration

✓ Whitelist specific origins

```
cors({  
  origin: ["http://localhost:5173"], // Only allow this  
  credentials: true,  
});
```

8. Rate Limiting (Not in BlogApp, but recommended)

```
import rateLimit from "express-rate-limit";  
  
const limiter = rateLimit({  
  windowMs: 15 * 60 * 1000, // 15 minutes  
  max: 100, // Limit each IP to 100 requests per windowMs  
});  
  
app.use("/api/", limiter);
```

Common Errors and Solutions

1. "Cannot set headers after they are sent"

Cause: Sending response multiple times

Problem:

```
export const getUser = async (req, res) => {
  let user = await userModel.findById(req.params.id);
  if (!user) {
    res.status(404).json({ message: "Not found" });
  }
  res.status(200).json({ user }); // ❌ Sends again!
};
```

Solution:

```
export const getUser = async (req, res) => {
  let user = await userModel.findById(req.params.id);
  if (!user) {
    return res.status(404).json({ message: "Not found" }); // ✅ return
  }
  res.status(200).json({ user });
};
```

2. "req.body is undefined"

Cause: Missing express.json() middleware**Solution:**

```
app.use(express.json()); // Add this before routes
```

3. "MongooseError: Operation buffering timed out"

Cause: Database not connected**Solution:**

```
// Make sure this is called and completes
await connectDB();
```

4. "ValidationError: email is required"

Cause: Missing required field**Solution:**

```
// Make sure all required fields are sent
await userModel.create({
  name: "John",
  email: "john@example.com", // Don't forget this
  password: "123",
});
```

5. "CastError: Cast to ObjectId failed"

Cause: Invalid ID format

Solution:

```
// Check ID format before querying
if (!mongoose.Types.ObjectId.isValid(id)) {
  return res.status(400).json({ message: "Invalid ID" });
}
```

6. "JsonWebTokenError: invalid token"

Cause: Invalid or tampered JWT

Solution:

```
// Make sure token is valid and SECRET_KEY matches
jwt.verify(token, process.env.SECRET_KEY);
```

7. "CORS Error"

Cause: Origin not allowed

Solution:

```
app.use(
  cors({
    origin: ["http://localhost:5173"], // Add your frontend URL
    credentials: true,
  })
);
```

8. "E11000 duplicate key error"

Cause: Unique constraint violation

Solution:

```
// Check if exists before creating
let existing = await userModel.findOne({ email });
if (existing) {
  return res.status(409).json({ message: "Email already exists" });
}
```

💡 Tips for Understanding the Code

1. Follow the Flow

Start from app.js and follow imports:

```
app.js → routes → controllers → models → database
```

2. Read Middleware Order

Middleware runs in order they're declared:

```
app.use(cors()); // 1st
app.use(cookieParser()); // 2nd
app.use(express.json()); // 3rd
app.use("/api/users", userRoutes); // 4th
```

3. Trace a Single Request

Pick one route and trace it completely:

```
POST /api/blogs/add
→ cors → cookieParser → express.json()
→ blogRoutes → authentication → addBlog
→ blogModel.create() → MongoDB
→ Response
```

4. console.log() is Your Friend

Add logs to understand flow:

```
export const addBlog = async (req, res) => {
  console.log("1. Received request");
  console.log("2. Body:", req.body);
  console.log("3. User:", req.myUser);

  let blog = await blogModel.create({...});
  console.log("4. Created blog:", blog);

  res.json({ blog });
};
```

5. Test with Postman/Thunder Client

- Test each endpoint individually
- Check request/response

- Understand what data is needed

6. Read Error Messages Carefully

```
ValidationError: User validation failed: email: Path `email` is required.  
                                     ↑  
                               Missing field
```

7. Understand Async/Await

```
// This waits for completion  
let user = await userModel.findById(id);  
console.log(user); // Runs after user is fetched  
  
// This doesn't wait  
let userPromise = userModel.findById(id);  
console.log(userPromise); // Logs: Promise { <pending> }
```

Learning Roadmap

For Beginners:

1. Understand JavaScript basics:

- Variables, functions, arrays, objects
- Promises, async/await
- ES6 features (arrow functions, destructuring)

2. Learn Node.js basics:

- What is Node.js?
- npm and package.json
- Module system (import/export)

3. Learn Express.js:

- Routing
- Middleware
- Request/Response

4. Learn MongoDB:

- CRUD operations
- Schema design
- Mongoose ODM

5. Understand this BlogApp:

- Start with simple routes (GET /all)
- Then protected routes (POST /add)
- Finally authentication flow

Next Steps:

1. Add features to BlogApp:

- Comments on blogs
- Like/unlike blogs
- User profiles
- Image uploads

2. Improve security:

- Rate limiting
- Input sanitization
- Helmet.js (security headers)

3. Add frontend:

- React.js
- Connect to this backend
- State management

4. Deploy:

- MongoDB Atlas (cloud database)
 - Heroku/Render (backend hosting)
 - Vercel/Netlify (frontend hosting)
-

Summary

BlogApp Architecture:

```

Client (Browser)
  ↓
  ↓ HTTP Request (with cookies)
  ↓
Express Server (app.js)
  ↓
  ↳ Middleware (CORS, cookieParser, express.json)
  ↓
  ↳ Routes (user.routes.js, blog.routes.js)
  ↓
  ↳ Authentication Middleware (auth.middleware.js)
  ↓
  ↳ Controllers (user.controller.js, blog.controller.js)
  ↓
  ↳ Models (user.model.js, blog.model.js)
  ↓
MongoDB Database
  ↓
  ↓ Response
  ↓
Client receives data
  
```

Key Technologies:

1. **Express.js** - Web framework
2. **MongoDB** - Database
3. **Mongoose** - ODM for MongoDB
4. **JWT** - Authentication tokens
5. **bcryptjs** - Password hashing
6. **CORS** - Cross-origin requests
7. **cookie-parser** - Cookie management
8. **dotenv** - Environment variables

Main Concepts:

✓ RESTful API design ✓ Middleware pattern ✓ Authentication & Authorization ✓ Password hashing ✓
 JWT tokens ✓ MongoDB relationships (referencing) ✓ Error handling ✓ Async/await ✓ CORS
 configuration



Quick Reference

Environment Variables (.env)

```
PORT=9000
MONGODB_URL=mongodb://localhost:27017/blogAPP
SECRET_KEY=your-secret-key-here
```

Start Application

```
npm install      # Install dependencies
npm start        # Start server
# or
npm run dev      # If using nodemon
```

Test Endpoints

Register:

```
POST /api/users/add
Body: { "name": "John", "email": "john@example.com", "password": "123456" }
```

Login:

```
POST /api/users/login
Body: { "email": "john@example.com", "password": "123456" }
```

Create Blog (needs login):

```
POST /api/blogs/add
Cookie: token=...
Body: { "title": "My Blog", "description": "Blog content here..." }
```

Get All Blogs:

```
GET /api/blogs/all
```



Conclusion

This BlogApp is a complete full-stack backend application demonstrating:

- User authentication with JWT
- CRUD operations for users and blogs
- Security best practices
- Error handling
- Database relationships
- RESTful API design

By understanding this application, you'll have a solid foundation in Node.js, Express, MongoDB, and backend development!

Happy Learning! 🚀